

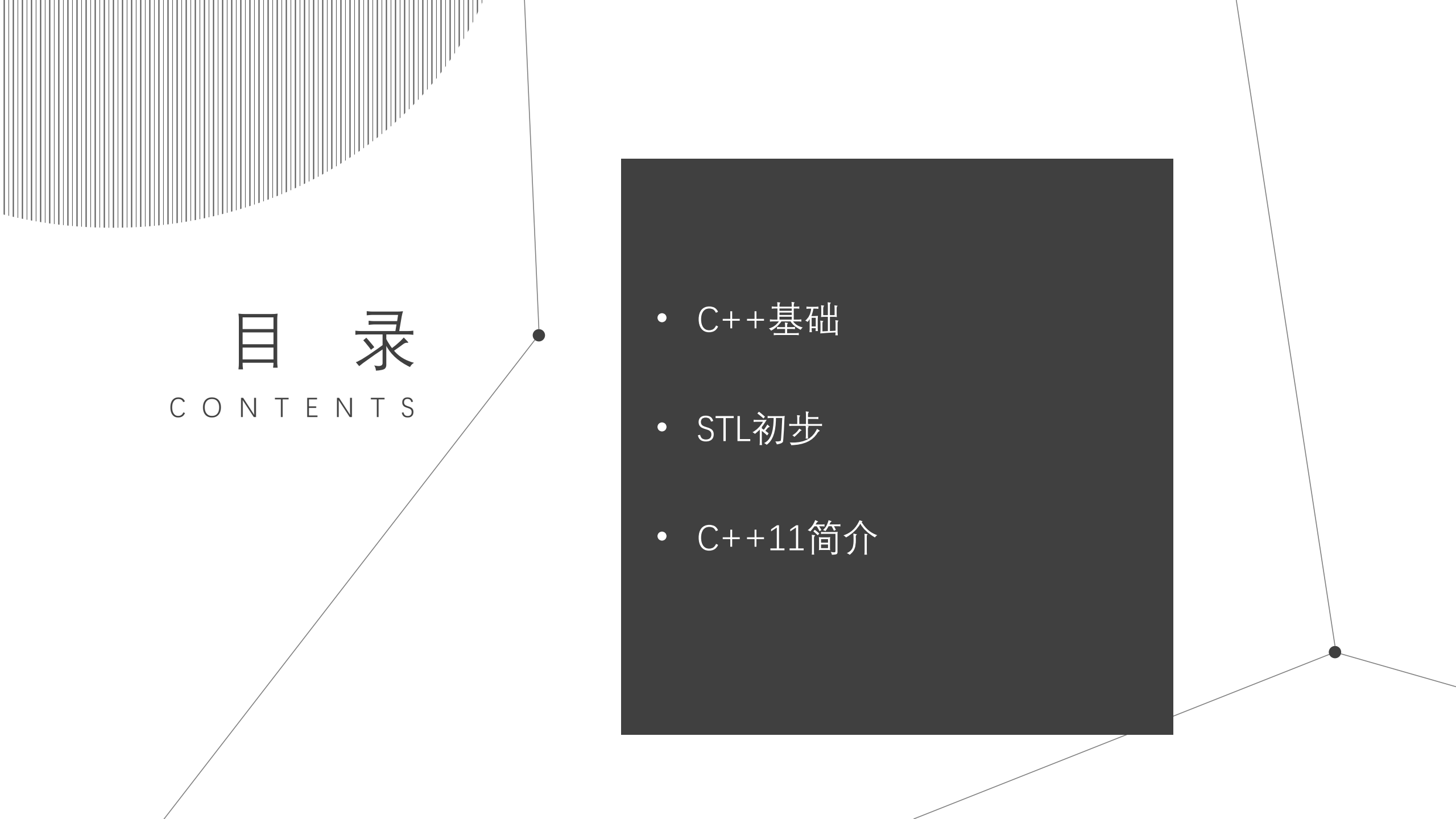
# C++基础串讲



串讲人：胡湘鹏



日期：2021- 11- 06



# 目 录

C O N T E N T S

- C++基础
- STL初步
- C++11简介



PART 01

# C++基础

```
1 #include<cstdio>
2 int main(){
3     int a, b;
4     while(scanf("%d%d", &a, &b) == 2)
5         printf("%d\n", a + b);
6     return 0;
7 }
```

唯一的区别: `stdio.h` 变成了 `cstdio`  
`stdio.h` 依然存在, 但是推荐 `cstdio`  
类似的还有 `cstring` `cmath` `cctype` ...

```
1  #include<iostream>
2  #include<algorithm>
3  using namespace std;
4  typedef long long LL;
5  const int N = 110;
6  int a[N];
7  int main() {
8      ios::sync_with_stdio(false);
9      LL a, b;
10     while(cin >> a >> b) {
11         cout << min(a, b) << endl;
12     }
13     return 0;
14 }
```

新增两个C++特有的头文件

iostream: 提供输入输出流

algorithm: 提供一些常用函数

cin: 从标准输入中读取

cout: 输出到标准输出

ios...(false): 关闭与stdio的同步

min: 返回较小值

namespace: 名称空间

数组大小可以使用const声明的常数  
更推荐使用const而不是#define

# 引用

```
1 #include<iostream>
2 using namespace std;
3 void swapByValue(int a, int b) {
4     int t = a; a = b; b = t;
5 }
6 void swapByReferece(int& a, int& b) {
7     int t = a; a = b; b = t;
8 }
9 int main(){
10     int a = 1, b = 2;
11     swapByValue(a, b);
12     cout << a << " " << b << endl;
13     a = 1, b = 2;
14     swapByReferece(a, b);
15     cout << a << " " << b << endl;
16     return 0;
17 }
18 /*
19 1 2
20 2 1
21 */
```

C++中的引用是变量的“别名”，可以在一定程度上替代C中的指针。

例如，可以使用“传引用”在函数内直接修改实参。

传引用也可以简化代码，使用十分灵活。algorithm 头文件定义了swap函数，功能十分强大。

```
int Abs(int n){ return n > 0 ? n : -n; }  
double Abs(double n){ return n > 0.0 ? n : -n; }  
printf("%d\n%.2f", Abs(-1), Abs(-2.1));
```

C++ 允许相同名称的不同函数，通过参数的个数和类型来区分(注意不能通过返回值区分)。在调用函数时，如果有多个函数符合要求，会按照某种顺序进行匹配（不推荐）。

```
int Sum(int a = 0, int b = 0, int c = 0) {  
    return a + b + c;  
}  
printf("%d\n%d\n%d\n%d\n", Sum(), Sum(1), Sum(1, 2), Sum(1, 2, 3));
```

C++允许指定函数参数的默认值(从右往左的任意个连续的参数),调用时可以省略给定了默认值的参数。

```
struct Node{  
    int x, y;  
    bool operator < (const Node & b) const {  
        return x < b.x;  
    }  
}a[N];
```

括号中的 `const` 表示参数 `b` 对象不会被修改，最后的 `const` 表明调用函数对象不会被修改。

<https://www.cnblogs.com/alking1001/p/12003178.html>

<https://www.cnblogs.com/ECJTUACM-873284962/p/6771262.html>

```
struct Point{
    int x, y;
    Point(int u = 0, int v = 0) : x(u), y(v) {}
    Point operator + (const Point& A) const {
        return Point(x + A.x, y + A.y);
    }
};

int main()
{
    Point a, b(1, 2);
    Point c = a + b;
    cout << c.x << " " << c.y << endl;
    return 0;
}
```

构造函数：在声明变量时调用，将成员变量x初始化为参数u，成员变量y初始化为参数v。

函数重载：构造函数可以有多个。

运算符重载：重载+，可以方便计算坐标相加。

# 模板

```
1  #include<iostream>
2  using namespace std;
3  template <typename T>
4  void mySwap(T &a,T &b) {
5      T t = a;
6      a = b;
7      b = t;
8  }
9  int main(){
10     int a = 1,b = 2;
11     mySwap(a, b);
12     cout << a << " " << b << endl;
13     double c = 3.1,d = 4.2;
14     mySwap(c, d);
15     cout << c << " " << d << endl;
16     return 0;
17 }
```

模板可以让功能相同的函数或类可以对多种数据类型适用，而无需对每种类型分别实现。

比如函数的差异只是参数类型不同，或者某种数据结构的元素可能有多种类型，这个时候就可以用模板。

## 使用模板重写Point结构体

```
template <typename T>
struct Point{
    T x, y;
    Point(T u = 0, T v = 0) : x(u), y(v) {}
    Point<T> operator + (const Point<T>& A) const {
        return Point<T>(x + A.x, y + A.y);
    }
};

int main()
{
    Point<int> a, b(1, 2);
    Point<int> c = a + b;
    cout << c.x << " " << c.y << endl;
    Point<double> d, e(3.1, 4.2);
    Point<double> f = d + e;
    cout << f.x << " " << f.y << endl;
    return 0;
}
```

模板在工程中的应用范围广泛，功能十分强大，虽然算法课程中使用较少，但是仍需了解。

了解模板有助于更好地理解STL。



PART 02

# STL初步

STL是指C++的标准模板库（Standard Template Library）。

STL功能强大且十分好用，使用STL可以增强程序可读性，帮助简洁高效地编写程序。

本次讲解侧重于应用，不会从语言的专业视角讲解STL的实现细节。

容器：可以容纳各种数据类型的通用数据结构（类模板）。

迭代器：类似于指针，用于访问容器中元素。

算法：用于对容器中元素进行操作的函数模板。

在需要使用对应的容器时，一般需要包含对应的头文件。

# 排序

```
struct Node{
    int x, y;
    bool operator < (const Node & b) const {
        return x < b.x;
    }
}a[N];
int main()
{
    int n; cin >> n;
    for (int i = 1; i <= n; i++) {
        cin >> a[i].x >> a[i].y;
    }
    sort(a + 1, a + 1 + n);
    for (int i = 1; i <= n; i++) {
        cout << a[i].x << " " << a[i].y << endl;
    }
    return 0;
}
```

## sort(begin,end)

begin: 待排序部分首元素的地址

end: 待排序部分最后一个元素的后继元素的地址，比如排序数组a的下标从1到n，就写：

sort(a + 1, a + 1 + n);

默认升序排列

# 排序

```
struct Node{
    int x, y;
}a[N];
bool cmp(const Node& A, const Node& B) {
    return A.x < B.x;
}
int main()
{
    int n; cin >> n;
    for (int i = 1; i <= n; i ++){
        cin >> a[i].x >> a[i].y;
    }
    sort(a + 1, a + 1 + n, cmp);
    for (int i = 1; i <= n; i ++){
        cout << a[i].x << " " << a[i].y << endl;
    }
    return 0;
}
```

## sort(begin,end,cmp)

cmp: 排序函数，若不写则默认升序。返回值为bool类型，若第一个参数排在第二个参数前则返回 true，否则返回 false。

对于内置类型，如果想要降序排序可以写 std::greater<类型名>()

sort是不稳定的排序，如果想要稳定排序可以使用stable\_sort，用法相同

## 不定长数组：vector

```
1  #include<iostream>
2  #include<algorithm>
3  #include<vector>
4  using namespace std;
5  const int N = 110;
6  vector<int> a, b[N];
7  int main()
8  {
9      a.push_back(1); a.push_back(2);
10     cout << a.front() << " " << a.back() << endl;
11     vector<int>::iterator it;
12     for (it = a.begin(); it != a.end(); it++) {
13         cout << *it << endl;
14     }
15     a.pop_back();
16     cout << a.size() << endl;
17     a.clear();
18     cout << a.empty() << endl;
19     return 0;
20 }
```

vector 可以理解为变长数组，它的内部实现基于倍增思想。vector 支持随机访问，既可以像数组一样使用下标  $[i]$  取值。

vector 不是链表，不支持任意位置的  $O(1)$  插入，元素的增删一般在末尾进行。

## 集合：set

```
1 #include<iostream>
2 #include<algorithm>
3 #include<set>
4 using namespace std;
5 const int N = 110;
6 set<int> s;
7 int main() {
8     s.insert(1), s.insert(2), s.insert(5);
9     if(s.find(1) != s.end()) puts("True");
10    set<int>::iterator it;
11    it = s.lower_bound(2); cout << *it << endl;
12    it = s.upper_bound(2); cout << *it << endl;
13    s.erase(it);
14    cout << s.count(5) << endl;
15 }
```

头文件 set 主要包括 set 和 multiset 两个容器，前者的元素不能重复，而后者可以包含若干个相等的元素。

set 和 multiset 的内部实现是红黑树（一种平衡二叉查找树），它们都是有序集合。

同样支持 size() empty() clear() begin() end()

插入 insert(x) 复杂度  $O(\log n)$

set 和 multiset 的迭代器仅支持“++”和“--”，不支持随机访问。

## 映射: map

```
1 #include<iostream>
2 #include<algorithm>
3 #include<map>
4 using namespace std;
5 const int N = 110;
6 string str;
7 int n, m;
8 map<string, int> h;
9 int main() {
10     cin >> n >> m;
11     for (int i = 1; i <= n; i++) {
12         cin >> str; h[str]++;
13     }
14     for (int i = 1; i <= m; i++) {
15         cin >> str;
16         if(h.find(str) != h.end()) cout << h[str] << endl;
17         else cout << 0 << endl;
18     }
19     return 0;
20 }
```

map 容器是一个键值对 key-value 的映射，内部实现是一棵以 key 为关键字的红黑树。

key 和 value 可以是任意类型，但 key 的类型必须定义 “<” 运算符。

与 set 类似同样支持 size()  
empty() clear() begin() end()  
insert() erase() find()

[] 操作符

h[key] 返回 key 映射到 value 的引用，可以直接通过 h[key] 对 value 进行赋值、查询、修改操作。

STL还提供 3 种特殊的数据结构：栈、队列、优先队列。

栈：“先进后出”原则，有PUSH和POP两种操作。

STL的栈定义在<stack>头文件中，用“`stack<int> s`”的方式定义，`push()`和`pop()`实现入栈和出栈，`top()`获取栈顶元素（不删除）。

队列：“先进先出”原则。

STL的队列定义在<queue>头文件中，用“`queue<int> q`”的方式定义，`push()`和`pop()`实现入队和出队，`front()`获取队首元素（不删除）。

优先队列：一种抽象数据类型，行为有些像队列，但先出队的元素是优先级最高的元素。

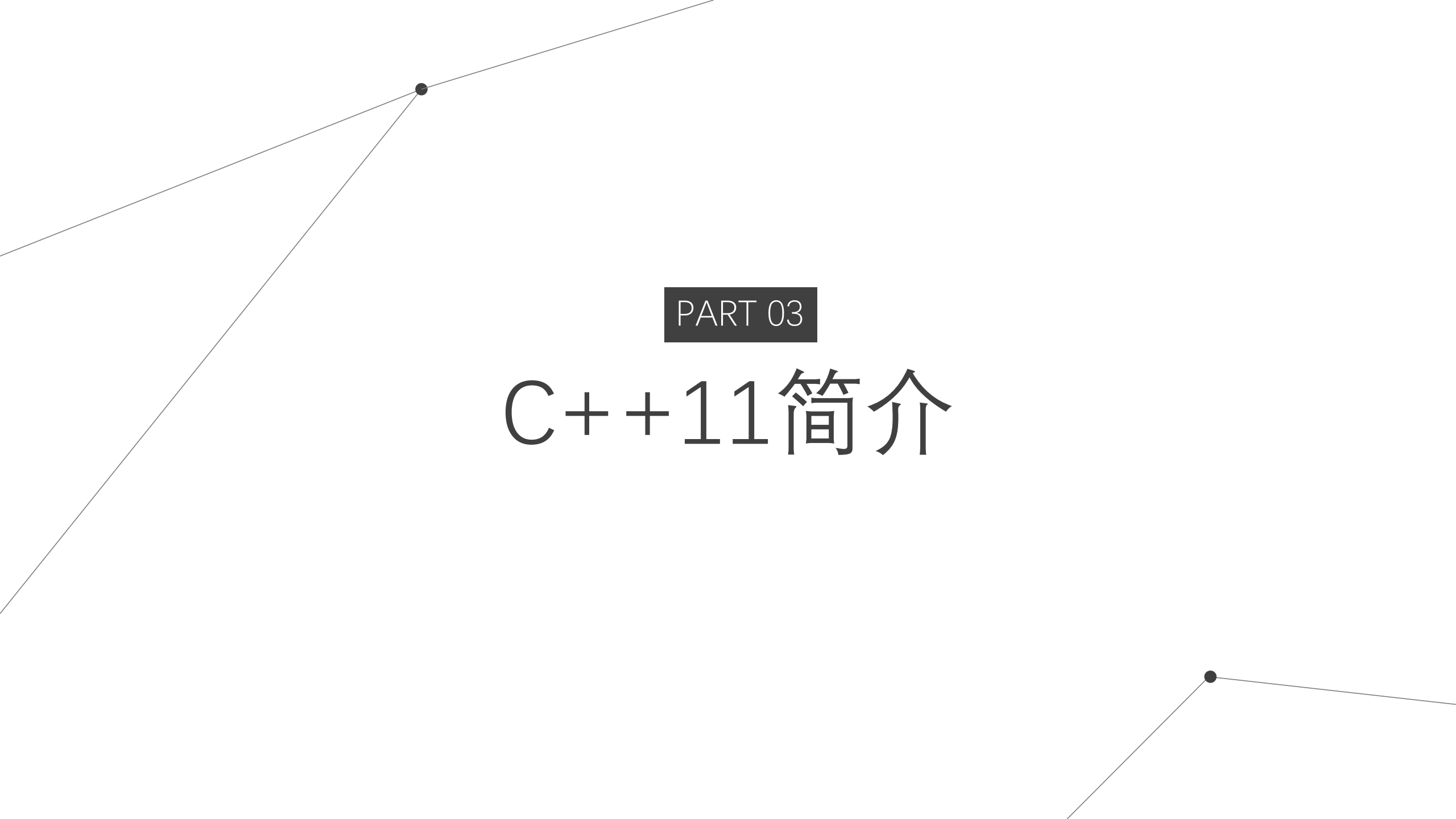
STL的优先队列定义在<queue>头文件中，用“priority\_queue<int> pq”的方式定义，push()和pop()实现入队和出队，top()获取队首元素（不删除）。

priority\_queue可以理解为一个大根二叉堆，其存储的元素类型必须定义“<”运算符，较大的元素会被放在堆顶。

内置数据类型可以本身可以比较大小，若使用自定义的结构体类型，则需要重载“<”运算符。

## 栈、队列、优先队列

```
struct Node {
    int x, y;
    bool operator < (const Node & b) const {
        return x < b.x || x == b.x && y < b.y;
    }
}a[N];
priority_queue<Node> pq;
int main() {
    int n; cin >> n;
    for (int i = 1; i <= n; i++) {
        cin >> a[i].x >> a[i].y;
        pq.push(a[i]);
    }
    while(pq.size()) {
        Node t = pq.top(); pq.pop();
        cout << t.x << " " << t.y << endl;
    }
}
```



PART 03

# C++11简介

C++11是C++的一个标准，相比C++98/03增加了很多新特性，这里简单讲一些平时写题可能用到的

开启C++11的方法:编译选项加 `-std=c++11`

accoding 的编译指令

c : `gcc main.c -O2 -std=c99 -DONLINE_JUDGE -o main`

c++ : `g++ main.cpp -O2 -std=c++11 -DONLINE_JUDGE -o main`

```
1 vector<int>::iterator it;  
2 for (it = a.begin(); it != a.end(); it++) {  
3     cout << *it << endl;  
4 }
```

```
1 for (auto it = a.begin(); it != a.end(); it++) {  
2     cout << *it << endl;  
3 }
```

```
1 for (auto i : a) {  
2     cout << i << endl;  
3 }
```

auto 可以在声明变量的时候根据变量初始值的类型自动为此变量选择匹配的类型。

时刻要注意auto并不是一个真正的类型，而仅仅是一个占位符，因此它不能使用一些以类型为操作数的操作符，如sizeof typeid等。

可以理解为执行一次的匿名函数(就是在调用函数的地方直接把函数写出来), 这样就不用单独声明和实现那些只执行很少次的函数了, 下面以sort的cmp函数为例:

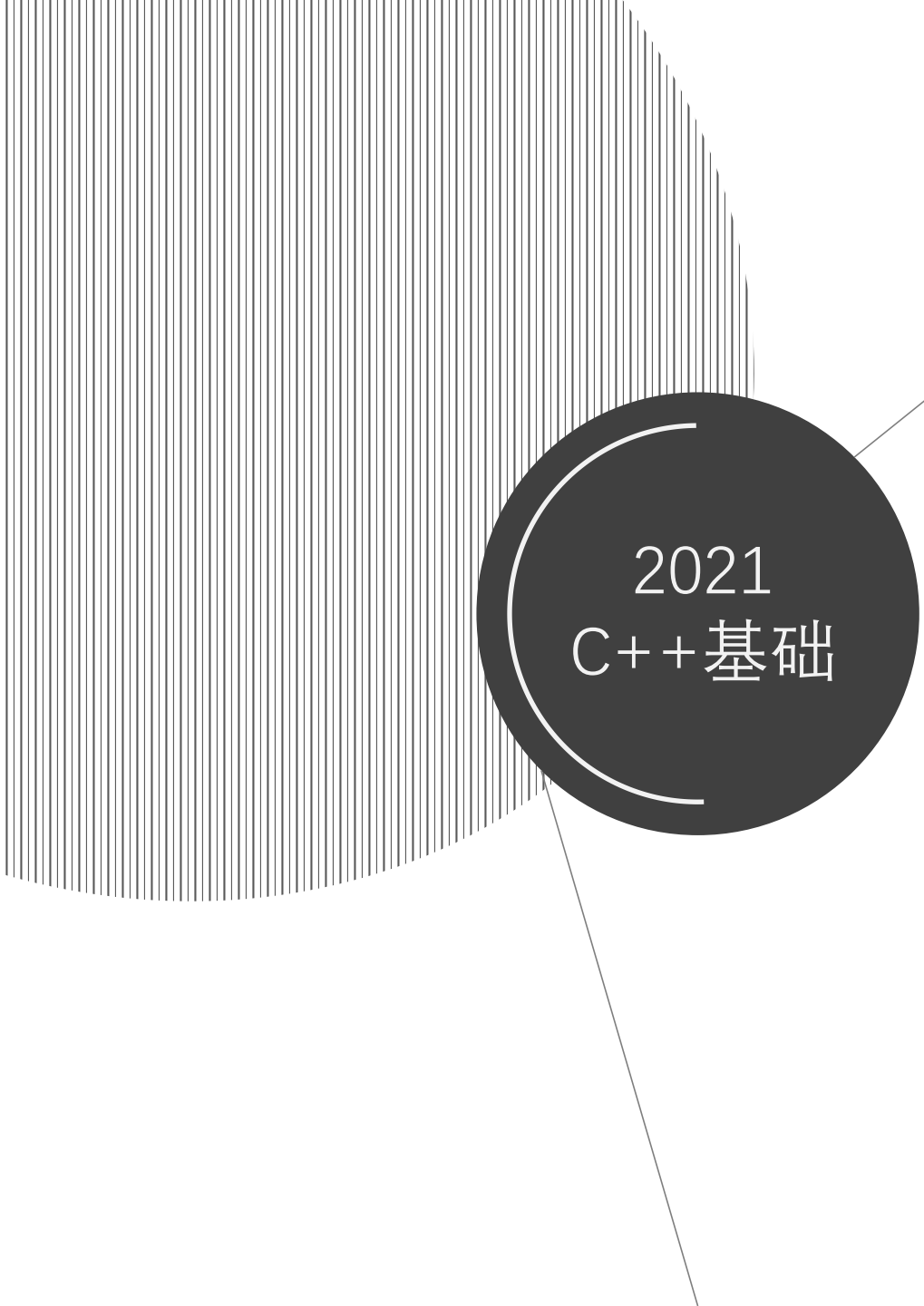
```
1 vector<int> v;  
2 sort(v.begin(),v.end(),[](const int &a,const int &b){return a > b;});
```

自 C++11 标准起，四种基于哈希实现的无序关联式容器正式纳入了 C++ 的标准模板库中，分别是：unordered\_set, unordered\_multiset, unordered\_map, unordered\_multimap。

采用哈希方式存储元素，内部元素不以任何特定顺序进行排序，所以访问无序关联式容器中的元素时，访问顺序也没有任何保证。

好处是无序关联式容器在平均情况下大多数操作（包括查找，插入，删除）都能在常数时间复杂度内完成，相较于关联式容器与容器大小成对数的时间复杂度更加优秀。

注意：需尽量避免滥用，一般仅在 set 或 map 有可能超时的情况下使用。



2021  
C++基础

THANK YOU

谢谢